Lab 10 — Operations and good practice

Theory — what happens after the `up`: observing, limiting, monitoring, hardening, distributing, cleaning up, and restarting on its own. The lab that turns "it runs on my machine" into "it runs, and we know why".

Objectives

- Observe a container in production: `logs`, `inspect`, `top`, `stats`, `events`.
- Set CPU and memory limits, understand the OOM killer and its link with the JVM.
- Write a `HEALTHCHECK` and know what Podman does with it (image format, systemd timers).
- Apply the basic hardening: non-root, read-only, capabilities, `no-new-privileges`.
- Choose a tag strategy, push to a private registry, clean up without breaking anything, start at boot with Quadlet.

---

1. Observing

A running container yields four sources of information, and each has its command:

Question	Command	What you read there
What is the application saying?	`podman logs -t --since 10m --tail 50 api`	its standard output and error, timestamped
How was it started?	`podman inspect api`	image, command, variables, mounts, limits, state, health
What is it doing now?	`podman top api`, `podman stats --no-stream`	processes, CPU and memory **actually** consumed
What happened?	`podman events --since 1h --filter container=api`	creation, start, stop, `health_status`, OOM, removal

Logs are the main source, and the rule from lab 06 applies: the application writes to **stdout/stderr**, the engine captures. Podman stores them by default in one file per container (`k8s-file` driver, path visible in `inspect` under `LogConfig.Path`); Docker uses `json-file`. Either way, a file that grows without limit eventually fills the disk: `--log-opt max-size=10m` (Docker) or `log_size_max` in `containers.conf` (Podman) cap it. In the workplace, an agent (Fluent Bit, Promtail, Filebeat) reads those files and ships them to a central platform (Loki, Elasticsearch) — the container itself knows nothing about it.

> **PODMAN**
> Without a daemon, there is no central event stream: `podman events` reads a journal written by each command (`events_logger`: `journald` on an Ubuntu with systemd, `file` otherwise). And `podman stats` requires **cgroup v2 delegation**: on WSL with `systemd=true`, that is the case; without systemd, Podman answers `Error: unable to get cgroup stats` or shows absurd aggregated values.

2. Limiting

Without a limit, a container can consume all the machine's memory and drag the others down with it. `--memory 256m` caps memory, `--cpus 0.5` limits to half a core, `--pids-limit 200` forbids

the fork bomb. All three are readable in `inspect` (`HostConfig.Memory`, `NanoCpus`, `PidsLimit`) and are enforced through cgroups.

> **→ LINUX**
> **cgroups v2** organise processes in a tree; each node carries control files: `memory.max` (ceiling), `memory.current` (usage), `cpu.max` (quota/period), `pids.max`. The container is a node in that tree, and `cat /sys/fs/cgroup/memory.max` **from inside the container** shows its own limit. When `memory.current` reaches `memory.max` and nothing more can be freed, the kernel triggers the **OOM killer**: it kills the hungriest process of the group, with `SIGKILL`. The container exits with `137`, and `inspect` says `OOMKilled: true`.

> **→ JAVA**
> Since Java 10, the JVM reads cgroups: it sees `256m`, not the machine's 32 GB, and sizes its heap accordingly. By default the heap takes **25 %** of the container's memory (half of it when the limit is small, under 512 MB); `-XX:MaxRAMPercentage=75` raises that to 75 %, leaving the rest to the *metaspace*, threads and native code. Passed through `JAVA_TOOL_OPTIONS` (lab 08), the setting does not touch the image. A JVM whose heap exceeds the limit does not throw `OutOfMemoryError`: it is killed by the kernel. The two numbers, container limit and heap percentage, are decided together.

3. Monitoring: the `HEALTHCHECK`

Lab 09 defined healthchecks in Compose. They can also be engraved into the image: `HEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -qO- http://localhost:8080/actuator/health || exit 1`. Every container started from that image then carries a `healthy` / `unhealthy` state in `podman ps`, without the deployer having to think about it — and Compose or Kubernetes can override it.

> **PODMAN**
> Two Podman-specific pitfalls. First: Buildah produces images in the **OCI** format by default, which has no `HEALTHCHECK` field; the instruction is **ignored**, with a single, easy-to-miss warning at build time (`HEALTHCHECK is not supported for OCI image format`), and `podman healthcheck run` answers `has no defined healthcheck`. `podman build --format docker` keeps it. Second: under rootless, the periodic check is run by a **systemd timer** per container; without a systemd session, the state stays `starting`. Docker has neither pitfall — but shares the rest: a healthcheck never **restarts** the container, it only changes its state. An orchestrator (or a `--restart` combined with `--health-on-failure=kill`, Podman-specific) acts on it.

4. Hardening

A container in production has five security settings to check, and none of them costs any code:

1. **Non-root in the image**: `USER 1000:1000` (lab 05). Under rootless, that `root` would already be your user; on a root Docker, it is the difference between a container escape and a compromised machine.
2. **Read-only filesystem**: `--read-only`, with `--tmpfs /tmp` for whatever must be written. An attacker can neither drop a binary nor modify the application.
3. **Minimal capabilities**: `--cap-drop ALL`, then `--cap-add` whatever is really missing (rarely anything for an API).
4. **No escalation**: `--security-opt no-new-privileges` forbids *setuid* binaries from gaining rights.
5. **Limits**: memory, CPU, PIDs (section 2) — a protection against denial of service, internal or external.

→ SECURITY

Capabilities split root's powers into about forty pieces: `CAP_CHOWN` (change an owner), `CAP_NET_BIND_SERVICE` (ports < 1024), `CAP_SYS_ADMIN` (almost everything)… A container receives about ten of them by default; `--cap-drop ALL` removes them all, and `chown` then answers `Operation not permitted` even for root. It is the principle of **least privilege** applied to the kernel: what is not granted cannot be exploited.

These options are found as such in Compose (`read_only`, `cap_drop`, `security_opt`) and in Kubernetes (`securityContext`). They replace neither image scanning (Trivy, Grype) nor base-image updates: a CVE in `glibc` is fixed by rebuilding, not by hardening.

5. Distributing: tags, digests, registry

A **tag** is a movable pointer; a **digest** is an immutable fingerprint (lab 02). The classic enterprise strategy: CI pushes `api:1.4.2` (semantic version), `api:1.4` and `api:1` (floating pointers), plus `api:git-a1b2c3d` (traceable to the commit); `latest` is never deployed; and the production deployment references the **digest** (`api@sha256:…`), the only guarantee that what runs is what was tested. A base tag (`postgres:16-alpine`) is pinned the same way before a production release.

The private registry is the hub: CI pushes to it, servers pull from it, scanners read it. `registry:2` is its minimal form; in the workplace you find Harbor, GitLab's registry, Nexus, Artifactory, or the cloud providers' — all speak the same `/v2/` API. A registry also serves as a **cache** for public images, so as not to depend on Docker Hub and its download limits.

PODMAN

Podman requires **TLS** by default, even for `localhost`: `podman push localhost:5000/api:1.0` fails with `http: server gave HTTP response to HTTPS client`. `--tls-verify=false` allows it for a test registry; for real, you declare the registry `insecure = true` in `registries.conf`, or better, give it a certificate. Docker has the same requirement, lifted by `insecure-registries` in `daemon.json`.

6. Cleaning up

A workstation or CI agent accumulates: intermediate `<none>` images (*dangling*), stopped containers, orphaned volumes, build cache. `podman system df` puts a number on each category and on what can be reclaimed. Then:

Command	Removes	Risk
`podman image prune`	dangling images	low: nothing named
`podman image prune -a`	**every** image without a container	high: removes `postgres`, `api:1.0`… to re-download or rebuild
`podman container prune`	every stopped container	medium: a container stopped "for later" disappears
`podman volume prune`	every volume no container uses	**very high**: a database between two `up`s is "unused" (lab 06)
`podman system prune`	stopped containers + unused networks + dangling images (+ `-a`, `--volumes`)	adds up the previous ones

PITFALL
`prune` reasons in "used by an existing container", never in "precious". Before deleting any volume, read `podman volume ls --filter dangling=true` and delete **by name**. On a workstation, `podman image prune` and `podman container prune` are almost always enough; `--volumes` and `-a` are reserved for a machine you accept to see emptied.

7. Starting at boot: Quadlet

Lab 03 showed it: without a daemon, `--restart` does not survive `wsl --shutdown`. Podman's answer is **Quadlet**: a file `~/.config/containers/systemd/api.container`, written like a systemd unit with a `[Container]` section (`Image=`, `PublishPort=`, `Environment=`, `Volume=`...), which `systemctl --user daemon-reload` turns into a service. `systemctl --user start api` launches it, `enable` activates it at boot, `journalctl --user -u api` shows its logs. The same files exist for networks, volumes, pods and Kubernetes files (`.kube`).

→ WINDOWS / WSL
A `--user` service only starts if your user session exists. For it to exist without a login: `loginctl enable-linger $USER`. And WSL only starts the distribution when a terminal is opened (or through a Windows scheduled task): "at boot" means "at the first `wsl` of the day". For a real server that is always on: Quadlet on a native Linux, or a Kubernetes cluster.

8. In the workplace: the place in a CI/CD chain

Everything you have seen fits in a *pipeline*: (1) `podman build` of an image, **once**, with `--format docker` if a `HEALTHCHECK` is expected; (2) a vulnerability **scan**, which blocks on critical CVEs; (3) `push` to the private registry with version and commit tags; (4) deployment to staging **by digest**, integration tests through Compose; (5) promotion of the **same digest** to production, by the orchestrator. Limits, hardening and the healthcheck are written in the manifests, not typed by hand. And day-to-day operations — centralised logs, metrics, alerts — rely on what the container exposes: `stdout`, `/actuator/health`, `/actuator/prometheus`.

Remember

- `logs`, `inspect`, `top` / `stats`, `events`: four questions, four commands; logs go to stdout and are centralised by an agent.
- `--memory` / `--cpus` / `--pids-limit` go through cgroups; exceeding memory = OOM killer, code `137`, `OOMKilled: true`; the JVM sizes its heap on the limit (`MaxRAMPercentage`).
- `HEALTHCHECK` in the image requires `--format docker` with Podman, and systemd timers under rootless; it signals, it does not restart.
- Harden: `USER`, `--read-only` + `--tmpfs`, `--cap-drop ALL`, `no-new-privileges`, limits — and scan the image.
- Version and commit tags, never `latest` in production, deploy by digest; private registry with TLS (`--tls-verify=false` only for testing).
- `prune` does not know "precious": `system df` first, delete volumes by name.
- Quadlet replaces `--restart` for starting at boot; on WSL, `enable-linger` and a running `wsl`.

Vocabulary

log driver: `k8s-file` (Podman) / `json-file` (Docker), one file per container. — **cgroups v2**: resource control tree (`memory.max` , `cpu.max` , `pids.max`). — **OOM killer**: kernel mechanism that kills a process when memory runs out. — **`MaxRAMPercentage`** : share of the container's memory given to the Java heap. — **`HEALTHCHECK`** : health command engraved in the image (docker format). — **capabilities**: split of root's privileges. — **`no-new-privileges`** : forbids escalation through setuid. — **floating tag / digest**: movable pointer / immutable fingerprint. — **private registry**: the company's image store, also a cache for public images. — **dangling**: image with neither tag nor container. — **Quadlet**: `.container` / `.pod` / `.kube` files turned into systemd units. — **linger**: systemd user session kept alive without a login.